



**Universidad Nacional
Autónoma de México**



Facultad de Ingeniería

**Laboratorio de Arquitectura y Organización
de Computadoras**

Grupo: 07 - Semestre: 2027-1

Reporte Practica 3

Construcción de Máquinas de estados Usando Memorias
Direccionamiento por Trayectoria

Fecha de Entrega

25 de Septiembre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando memorias con el método de direccionamiento por trayectoria.

2. Desarrollo

Se implementó el control de una *llenadora industrial* mediante una máquina de estados finitos realizada con una memoria ROM. El proceso considera tres entradas: el botón de inicio X , el sensor de tapa cerrada Y y el sensor de agua llena Z . Las cuatro salidas son la válvula de llenado de agua V , la bomba de drenaje B , el indicador de alarma P y la señal de control del motor M . De acuerdo con la carta ASM proporcionada, V y B son activas en alto, mientras que P y M se expresan con lógica negada; por ello, un cero en estas últimas señales activa la función correspondiente.

El diseño se construyó por direccionamiento por trayectoria. El registro conserva el estado presente; el bloque concatenador une dicho estado con las entradas para formar la dirección de la ROM; y la palabra leída se divide en liga (estado siguiente) y salidas. De esta manera, cada trayectoria de la carta ASM queda codificada directamente en una localidad de memoria, sin requerir simplificación mediante mapas de Karnaugh.

2.1. Carta ASM

La Figura 1 reproduce en L^AT_EX la carta ASM de la llenadora industrial. Los bloques rectangulares representan salidas incondicionales; los rombos, decisiones de entrada; y los óvalos, salidas condicionales. Las etiquetas 0 y 1 indican la trayectoria elegida en cada decisión.

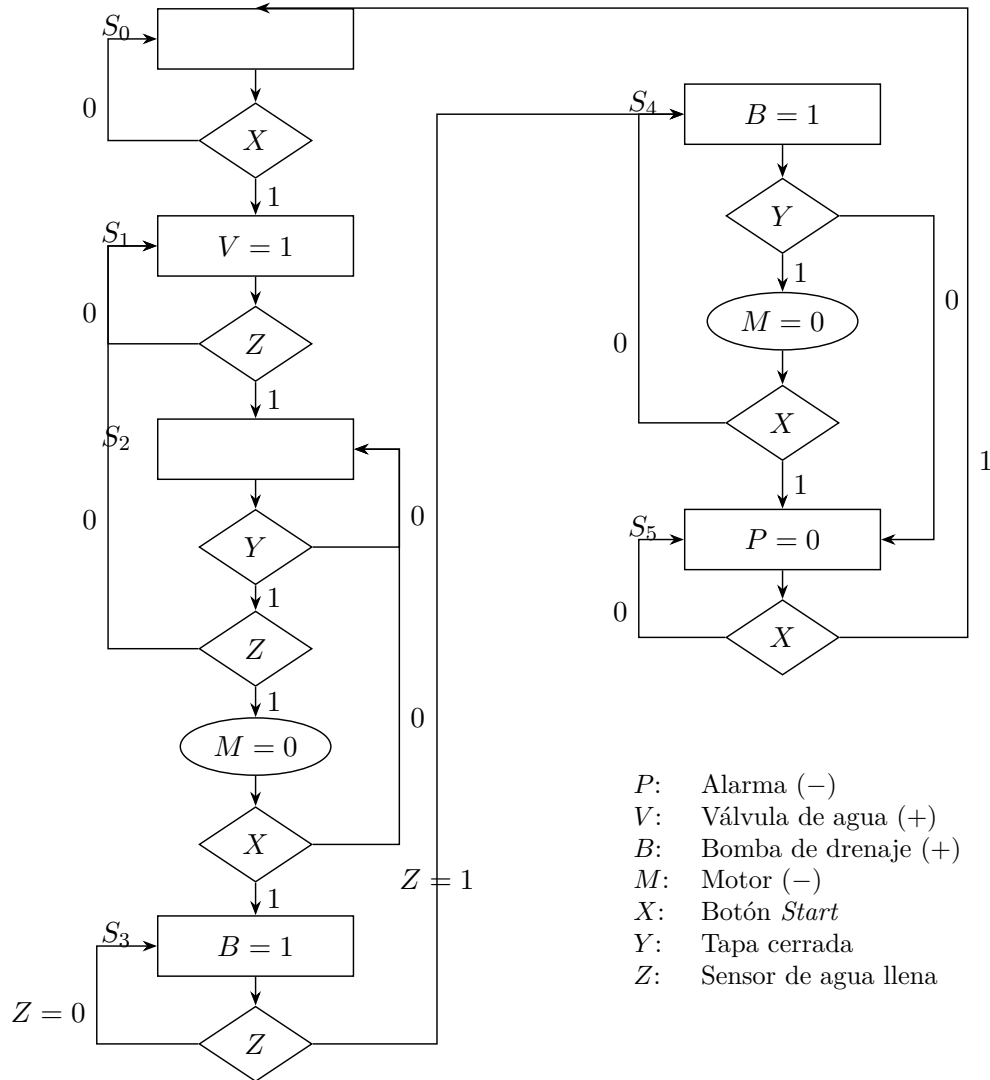


Figura 1: Carta ASM de la llenadora industrial.

2.2. Dimensionamiento de la memoria

La máquina tiene $C = 6$ estados (S_0 a S_5), tres entradas y cuatro salidas. Se requieren $q = \lceil \log_2 C \rceil = 3$ bits para codificar el estado. En el direccionamiento por trayectoria, la dirección combina el estado presente con las entradas; la palabra de memoria contiene la liga

y las salidas. Por tanto:

$$\underbrace{n}_{\text{bits de dirección}} = q + I = 3 + 3 = 6, \quad \underbrace{m}_{\text{bits por palabra}} = q + O = 3 + 4 = 7.$$

La ROM requerida es de $2^n \times m = 2^6 \times 7 = 64 \times 7$ bits, con una capacidad total de 448 bits. La dirección se organiza como Q2 Q1 Q0 X Y Z y el contenido de cada palabra como L2 L1 L0 V B P M.

2.3. Tabla de estados, entradas y salidas

La Tabla 1 reúne las trayectorias de la carta ASM. El asterisco indica una entrada no relevante para la trayectoria. Las columnas *liga* y *salidas* se escriben en el mismo orden usado en la ROM: L2L1L0 y VBPM, respectivamente.

Tabla 1: Tabla de transición y salidas obtenida de la carta ASM.

Estado	$Q_2Q_1Q_0$	X	Y	Z	Liga	Salidas
S_0	000	0	*	*	000	0011
	000	1	*	*	001	0011
S_1	001	*	*	0	001	1011
	001	*	*	1	010	1011
S_2	010	*	0	*	010	0011
	010	*	1	0	001	0011
	010	0	1	1	010	0010
	010	1	1	1	011	0010
S_3	011	*	*	0	011	0111
	011	*	*	1	100	0111
S_4	100	*	0	*	101	0111
	100	0	1	*	100	0110
	100	1	1	*	101	0110
S_5	101	0	*	*	101	0001
	101	1	*	*	000	0001
Estados no definidos	110, 111	*	*	*	000	0011

Las codificaciones 110 y 111 no corresponden a un estado de la carta ASM. Para recuperar el funcionamiento ante cualquiera de ellas, las tres entradas se consideran *don't cares* ($X = Y = Z = *$) y la liga se fija en 000. Así, sin importar la combinación de entrada, la máquina regresa a S_0 con las salidas inactivas VBPM=0011. Esta condición corresponde a las direcciones 48 a 63 de la ROM.

2.4. Implementación en VHDL

El proyecto se organizó como una máquina de estados implementada por memoria. El flujo de datos inicia en el registro: en cada flanco ascendente de reloj guarda la liga que recibió en el ciclo anterior y la presenta como **edopresente**. En paralelo, el concatenador une los tres bits del estado presente con las entradas XYZ; este vector de seis bits selecciona una palabra de la ROM. La palabra leída se divide en dos campos: los bits [6 : 4] forman la liga que volverá al registro en el siguiente flanco, y los bits [3 : 0] se envían de inmediato a las salidas VBPM. Por tanto, el recorrido completo es

`registro → concatenador → rom → div_datos → registro.`

Registro de estado. El módulo **registro** contiene la señal interna **internal_value**, inicializada en 000. Si **reset='0'**, el primer proceso fuerza inmediatamente dicha codificación, por lo que S_0 es el estado seguro de arranque. En cualquier otro caso, el bloque sólo actualiza el estado al detectar **rising_edge(clk)**; así se evita que cambios en las entradas modifiquen el estado entre flancos. El segundo proceso copia el valor interno a **edopresente** para que pueda emplearse como parte de la dirección.

Formación de dirección. **concatenador** es lógica combinacional: cada vez que cambia una entrada o el estado presente, ejecuta **dir <= edopresente & entradas**. Al colocar **edopresente** antes de **entradas**, sus bits son los más significativos. Por ello, todas las direcciones de S_0 comienzan con 000, las de S_1 con 001, y así sucesivamente; dentro de cada grupo, XYZ selecciona la trayectoria de la carta ASM.

ROM y división de palabra. En **rom.vhd**, el tipo **mem** declara 64 posiciones de siete bits. Cada asignación tiene la forma **"liga"& "salida"**; por ejemplo, las posiciones de S_0 escriben 000&0011 o 001&0011, según X . El proceso de lectura convierte **dir** de binario a entero mediante **conv_integer(unsigned(dir))** y entrega la palabra seleccionada en **data**. Después, **div_datos** toma **data(6 downto 4)** como liga y **data(3 downto 0)** como VBPM; por ello no necesita reloj ni memoria adicional. Para que la recuperación indicada en la última fila de la tabla quede implementada también en la ROM, las posiciones 48 a 63 deben almacenar 000&0011.

Acondicionamiento del botón. **clk_button** es un bloque auxiliar independiente de la ROM. Sus estados **STATE_START**, **STATE_COUNT** y **STATE_DONE** corresponden a espera, con-

teo y finalización. Al detectar `button='1'`, pasa a `STATE_COUNT`; mientras permanece ahí, incrementa cuenta en cada ciclo del reloj base y mantiene `div_clk='1'`. Cuando alcanza `MAX_CLK_COUNT` (25 millones), pasa por `STATE_DONE` y regresa a espera. De esta forma, una pulsación genera una sola transición ascendente en `div_clk`; mantener la señal alta durante el conteo evita que los rebotes del botón generen múltiples transiciones de inicio.

Listing 1: Módulo de registro del estado presente.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity registro is
7      port(
8          clk : in std_logic;
9          reset : in std_logic;
10         edosiguiente : in std_logic_vector(2 downto 0);
11         edopresente : out std_logic_vector(2 downto 0)
12     );
13 end registro;
14
15 architecture Behavioral of registro is
16
17     signal internal_value: std_logic_vector(2 downto 0) := B"000";
18
19     begin
20         process(clk, reset, edosiguiente)
21         begin
22             if reset = '0' then
23                 internal_value <= B"000";
24             elsif rising_edge(clk) then
25                 internal_value <= edosiguiente;
26             end if;
27         end process;
28
29         process(internal_value)
30         begin
31             edopresente <= internal_value;
32         end process;
33
34     end Behavioral;

```

Listing 2: Módulo que concatena el estado presente y las entradas para formar la dirección.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity concatenador is
7      port(
8          entradas: in std_logic_vector(2 downto 0);
9          edopresente: in std_logic_vector(2 downto 0);
10         dir: out std_logic_vector(5 downto 0)
11     );
12 end concatenador;
13
14 architecture Behavioral of concatenador is
15 begin
16     process(entradas, edopresente)

```

```

17     begin
18         dir <= edopresente & entradas; -- EdoPresente Bits mas significativos
19     end process;
20 end Behavioral;

```

Listing 3: Memoria ROM que codifica las ligas y las salidas.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity rom is
7      port(
8          dir: in std_logic_vector(5 downto 0); -- edopresente + entradas
9          data: out std_logic_vector(6 downto 0) -- Liga + Salida
10     );
11 end rom;
12
13 architecture Behavioral of rom is
14     type mem is array (0 to 63) of std_logic_vector(6 downto 0);
15     signal internal_mem: mem;
16
17     begin
18         -- Liga + Salida
19         --S0
20         internal_mem(0) <= "000" & "0011";
21         internal_mem(1) <= "000" & "0011";
22         internal_mem(2) <= "000" & "0011";
23         internal_mem(3) <= "000" & "0011";
24         internal_mem(4) <= "001" & "0011";
25         internal_mem(5) <= "001" & "0011";
26         internal_mem(6) <= "001" & "0011";
27         internal_mem(7) <= "001" & "0011";
28         --S1
29         internal_mem(8) <= "001" & "1011";
30         internal_mem(9) <= "010" & "1011";
31         internal_mem(10) <= "001" & "1011";
32         internal_mem(11) <= "010" & "1011";
33         internal_mem(12) <= "001" & "1011";
34         internal_mem(13) <= "010" & "1011";
35         internal_mem(14) <= "001" & "1011";
36         internal_mem(15) <= "010" & "1011";
37         --S2
38         internal_mem(16) <= "010" & "0011";
39         internal_mem(17) <= "010" & "0011";
40         internal_mem(18) <= "001" & "0011";
41         internal_mem(19) <= "010" & "0010";
42         internal_mem(20) <= "010" & "0011";
43         internal_mem(21) <= "010" & "0011";
44         internal_mem(22) <= "001" & "0011";
45         internal_mem(23) <= "011" & "0010";
46         --S3
47         internal_mem(24) <= "011" & "0111";
48         internal_mem(25) <= "100" & "0111";
49         internal_mem(26) <= "011" & "0111";
50         internal_mem(27) <= "100" & "0111";
51         internal_mem(28) <= "011" & "0111";
52         internal_mem(29) <= "100" & "0111";
53         internal_mem(30) <= "011" & "0111";
54         internal_mem(31) <= "100" & "0111";
55         --S4
56         internal_mem(32) <= "101" & "0111";

```

```

57     internal_mem(33) <= "101" & "0111";
58     internal_mem(34) <= "100" & "0110";
59     internal_mem(35) <= "100" & "0110";
60     internal_mem(36) <= "101" & "0111";
61     internal_mem(37) <= "101" & "0111";
62     internal_mem(38) <= "101" & "0110";
63     internal_mem(39) <= "101" & "0110";
64     --S5
65     internal_mem(40) <= "101" & "0001";
66     internal_mem(41) <= "101" & "0001";
67     internal_mem(42) <= "101" & "0001";
68     internal_mem(43) <= "101" & "0001";
69
70     internal_mem(44) <= "000" & "0001";
71     internal_mem(45) <= "000" & "0001";
72     internal_mem(46) <= "000" & "0001";
73     internal_mem(47) <= "000" & "0001";
74
75     process(dir)
76     begin
77         data <= internal_mem(conv_integer(unsigned(dir)));
78     end process;
79 end Behavioral;

```

Listing 4: Módulo que separa liga y salidas de la palabra de la ROM.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity div_datos is
7      port(
8          data: in std_logic_vector(6 downto 0);
9          liga: out std_logic_vector(2 downto 0);
10         salidas: out std_logic_vector(3 downto 0)
11     );
12 end div_datos;
13
14 architecture Behavioral of div_datos is
15 begin
16     process(data)
17     begin
18         liga <= data(6 downto 4);
19         salidas <= data(3 downto 0);
20     end process;
21 end Behavioral;

```

Listing 5: Módulo de acondicionamiento temporal del botón.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity clk_button is
7      port(
8          clk : in std_logic;
9          button : in std_logic;
10         div_clk : out std_logic
11     );
12 end clk_button;

```

```
13
14 architecture Behavioral of clk_button is
15
16 CONSTANT STATE_START: std_logic_vector(1 downto 0) := "00";
17 CONSTANT STATE_COUNT: std_logic_vector(1 downto 0) := "01";
18 CONSTANT STATE_DONE: std_logic_vector(1 downto 0) := "10";
19
20 CONSTANT MAX_CLK_COUNT: std_logic_vector(27 downto 0) := X"17D7840"; -- 25 millones de
    pulsos
21
22 signal state: std_logic_vector(1 downto 0) := "00";
23 signal cuenta: std_logic_vector(27 downto 0) := X"0000000";
24
25 begin
26     process(clk, button)
27     begin
28         if rising_edge(clk) then
29             case state is
30                 WHEN STATE_START =>
31                     if button = '1' then
32                         state <= STATE_COUNT;
33                     end if;
34                 WHEN STATE_COUNT =>
35                     if cuenta >= MAX_CLK_COUNT then
36                         state <= STATE_DONE;
37                     end if;
38                 WHEN STATE_DONE =>
39                     state <= STATE_START;
40                 WHEN others =>
41                     state <= STATE_START;
42             end case;
43         end if;
44     end process;
45
46     process(clk)
47     begin
48         if rising_edge(clk) then
49             if state = STATE_COUNT then
50                 cuenta <= cuenta + 1;
51             else
52                 cuenta <= X"0000000";
53             end if;
54         end if;
55     end process;
56
57     process(state)
58     begin
59         if state = STATE_COUNT then
60             div_clk <= '1';
61         else
62             div_clk <= '0';
63         end if;
64     end process;
65 end Behavioral;
```

3. Simulaciones

Las simulaciones verifican el recorrido de los seis estados y la correspondencia entre cada combinación Q2Q1Q0XYZ y el contenido L2L1L0VBPM de la ROM. Para permitir el funciona-

miento normal del registro, **reset** se mantiene en uno durante la simulación; el reinicio del módulo es activo en bajo. Las entradas se probaron primero con combinaciones aleatorias y, en el último tramo, se forzaron a 111 para recorrer de forma visible todos los estados de la carta ASM.

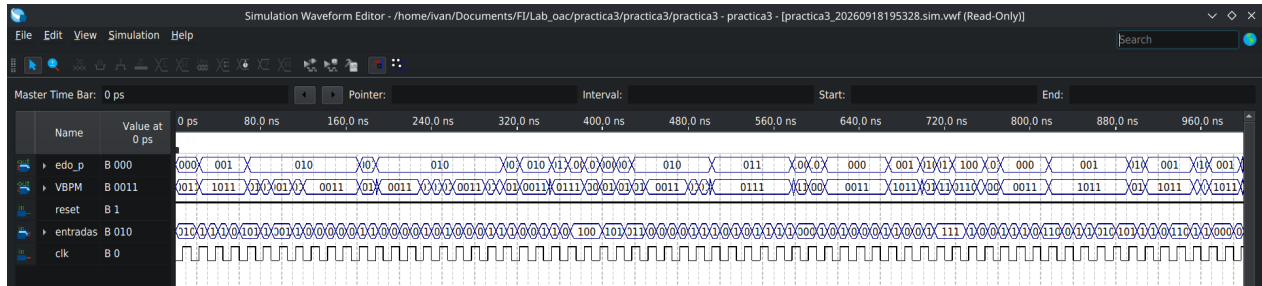


Figura 2: Primera sección de la simulación. Se mantienen **reset=1** y combinaciones aleatorias en **entradas**.

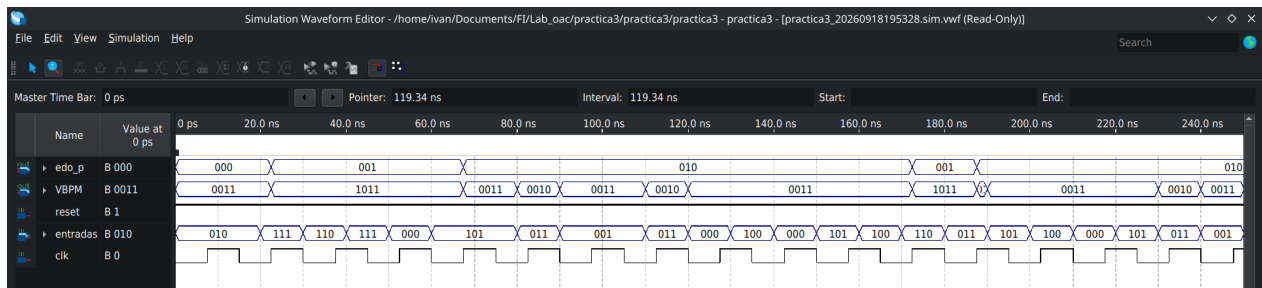


Figura 3: Detalle de la respuesta de **edo_p** y **VBPM** ante cambios aleatorios en las entradas.

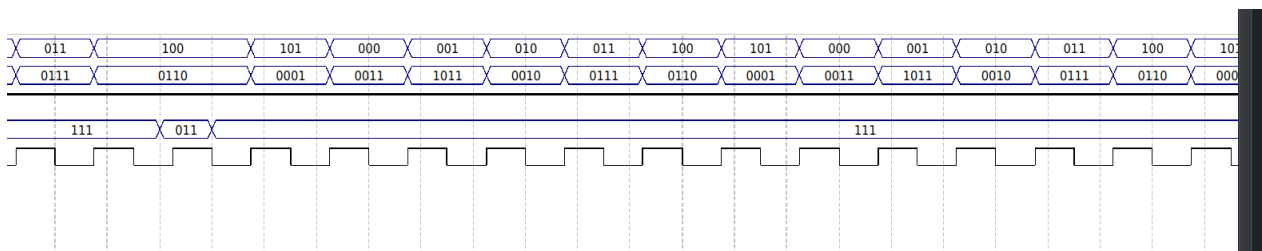


Figura 4: Tramo final forzado con **entradas=111**; se observa el recorrido completo por los estados de la carta ASM.

4. Resultados

Las Figuras 5 y 6 muestran el diagrama de bloques implementado en Quartus. El reloj y el botón ingresan al bloque **clk_button**; el registro entrega el estado presente, el concatenador

genera la dirección de seis bits y la ROM produce la palabra de siete bits. El bloque `div_datos` distribuye la liga hacia el registro y las salidas VBPM hacia los pines de la tarjeta.

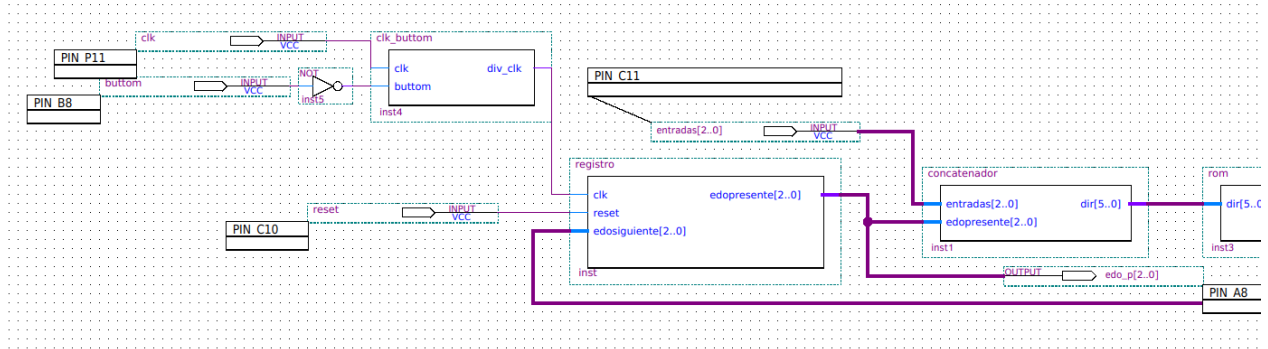


Figura 5: Interconexión del reloj, el registro de estado, las entradas y el generador de dirección.

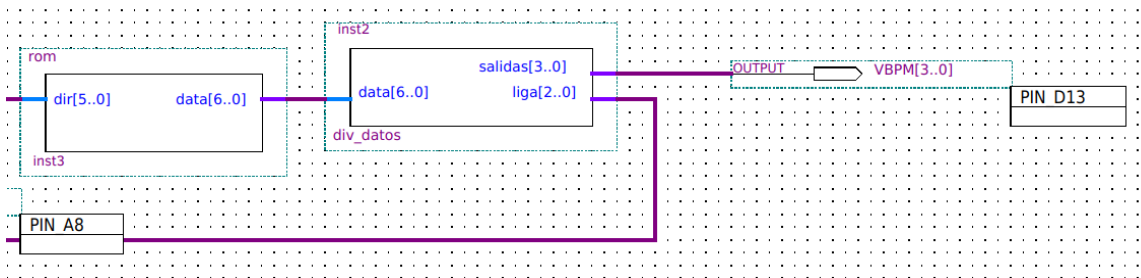
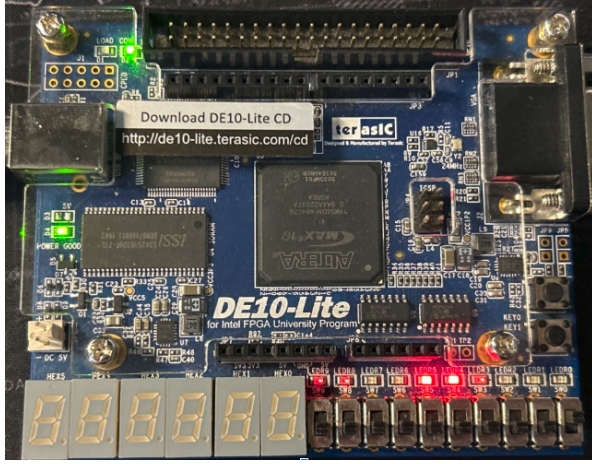


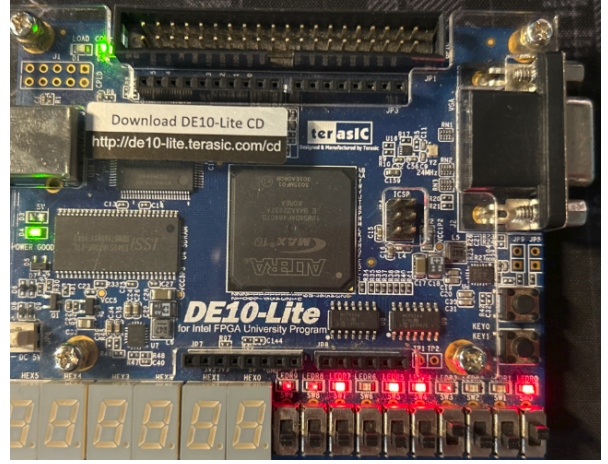
Figura 6: ROM y división de la palabra de datos en liga y salidas VBPM.

4.1. Evidencia en la tarjeta FPGA

Las fotografías se organizaron de acuerdo con las filas de la Tabla 1. En cada caso se seleccionó la combinación de entradas indicada y se verificaron los LEDs asociados a `edo_p` y `VBPM`. De esta forma, las evidencias físicas permiten contrastar las ligas y las salidas programadas en la ROM.

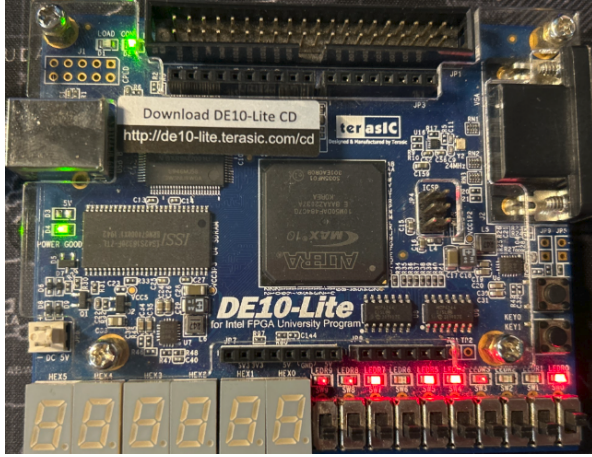


(a) $S_0, X = 0$: se conserva el estado inicial.

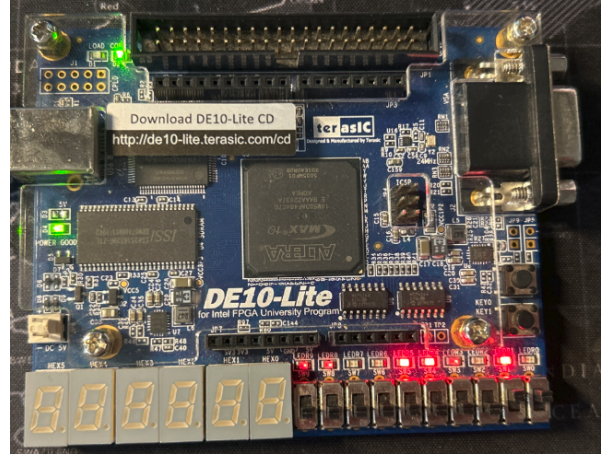


(b) $S_0, X = 1$: liga hacia S_1 .

Figura 7: Evidencia en tarjeta para las trayectorias del estado S_0 .

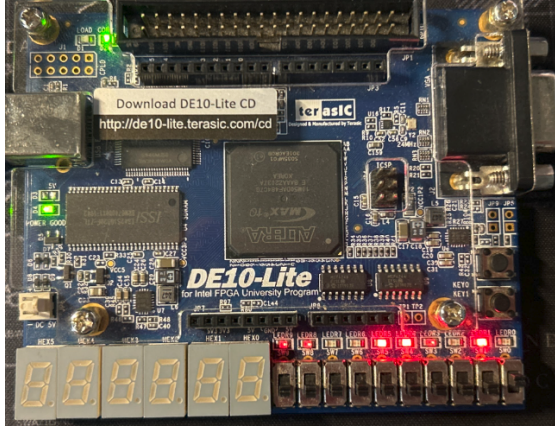


(a) $S_1, Z = 0$: permanece en llenado.

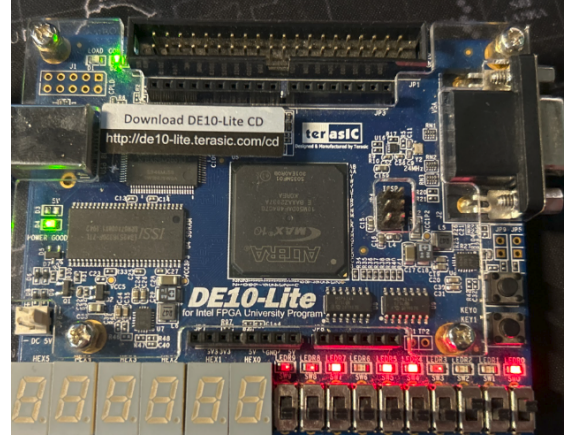


(b) $S_1, Z = 1$: liga hacia S_2 .

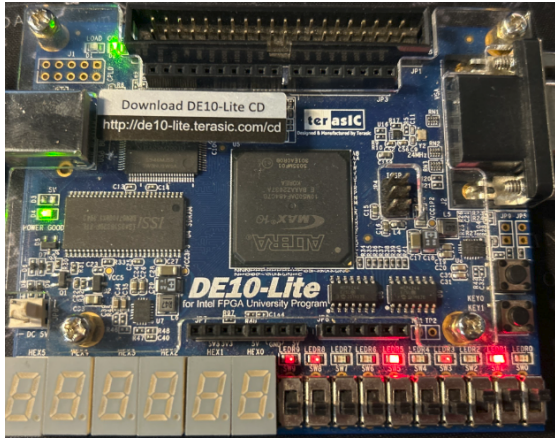
Figura 8: Evidencia en tarjeta para las trayectorias del estado S_1 .



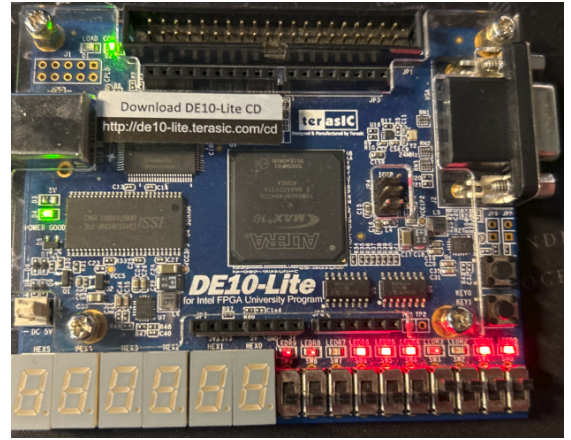
(a) S_2 , $Y = 0$.



(b) S_2 , $Y = 1$, $Z = 0$: liga hacia S_1 .

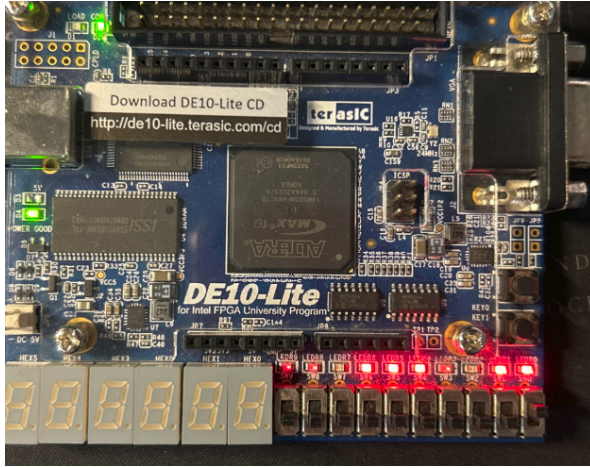


(c) S_2 , $X = 0$, $Y = 1$, $Z = 1$.

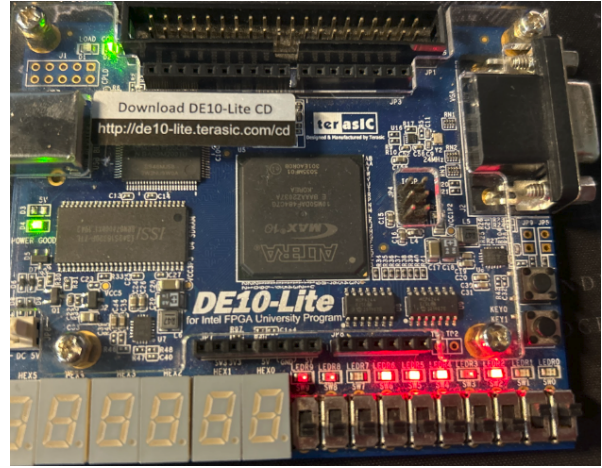


(d) S_2 , $X = 1$, $Y = 1$, $Z = 1$: liga hacia S_3 .

Figura 9: Evidencia en tarjeta para las cuatro trayectorias del estado S_2 .

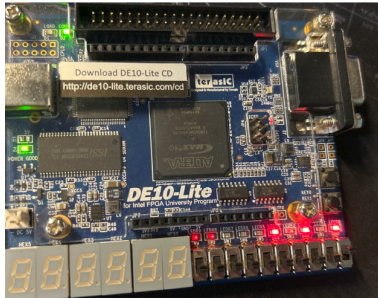


(a) $S_3, Z = 0$: permanece en drenaje.

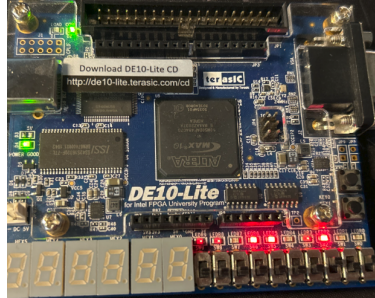


(b) $S_3, Z = 1$: liga hacia S_4 .

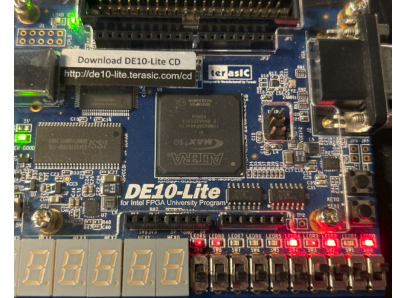
Figura 10: Evidencia en tarjeta para las trayectorias del estado S_3 .



(a) $S_4, Y = 0$.

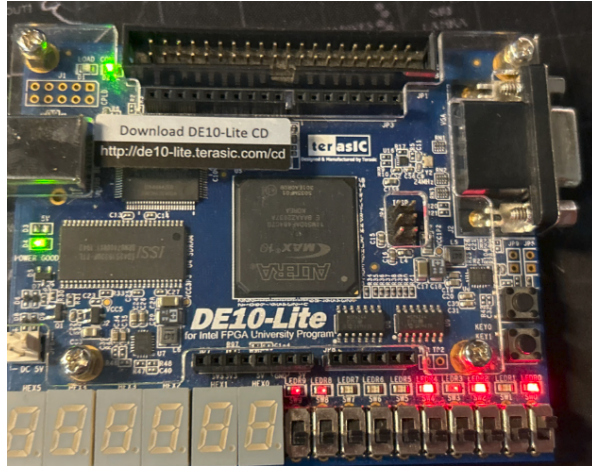


(b) $S_4, X = 0, Y = 1$.

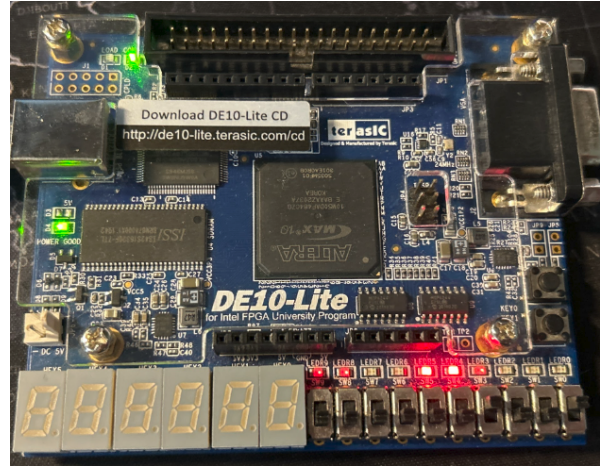


(c) $S_4, X = 1, Y = 1$.

Figura 11: Evidencia en tarjeta para las trayectorias del estado S_4 ; Z es indiferente en este estado.



(a) S_5 , $X = 0$: mantiene la alarma.



(b) S_5 , $X = 1$: liga hacia S_0 .

Figura 12: Evidencia en tarjeta para las trayectorias del estado S_5 .

5. Conclusión

El direccionamiento por trayectoria permitió traducir la carta ASM a una ROM de 64×7 bits de manera directa: los seis bits de dirección especifican el estado presente y las entradas, mientras que los siete bits almacenados determinan el estado siguiente y las salidas. La división del diseño en registro, concatenador, ROM y divisor de datos hace visible cada etapa del método y facilita contrastar la implementación VHDL con la tabla de direccionamiento. Las simulaciones, con `reset=1`, y la evidencia en la tarjeta FPGA mostraron las transiciones previstas por la carta ASM, incluyendo el recorrido completo al forzar `XYZ=111`.